Time to escape

Kirt Onthank

2026-07-27

Contents

Setup

Loading Libraries

This section loads the R packages needed for the entire analysis:

- **xlsx**: Reads the Excel data file containing time-to-escape measurements for each trial (P1–P5).
- **dplyr**: Provides data manipulation functions (`arrange`, `group_by`, `mutate`, `filter`, etc.).
- **coxme**: Fits mixed-effects Cox proportional hazards models — the primary survival analysis tool. This allows us to include random effects (e.g., individual squid nested within tank) to account for repeated measures on the same animals.
- **survival**: Provides the `Surv()` function for creating survival objects and `survfit()` for Kaplan–Meier curves.
- **readODS**: Reads the OpenDocument Spreadsheet (`.ods`) file containing distance/path-length measurements.
- **lme4**: Fits generalized linear mixed models (GLMMs), used later for analyzing escape probability (binary outcome).
- **lmerTest**: Provides p-values for linear mixed models (though not heavily used in this script).
- **kableExtra**: Formats summary tables with `kable()` for publication-quality output in the PDF.

```r
library(xlsx)
library(dplyr)
library(coxme)
library(survival)
library(readODS)
library(lme4)
library(lmerTest)
library(kableExtra)
```

Loading data

Data are loaded from the Excel file `E. berryi data.xlsx`, which contains separate sheets for each experimental procedure (P1 through P5). Each sheet holds trial-by-trial observations for individual squids: the time (in seconds) taken to locate and swim through the escape hole in the open-field tank.

```r
P1=read.xlsx("E. berryi data.xlsx",sheetIndex = "P1")
P2=read.xlsx("E. berryi data.xlsx",sheetIndex = "P2")
P3=read.xlsx("E. berryi data.xlsx",sheetIndex = "P3")
P4=read.xlsx("E. berryi data.xlsx",sheetIndex = "P4")
P5=read.xlsx("E. berryi data.xlsx",sheetIndex = "P5")
```

After loading, a `Trial` column is added to each data frame to identify which procedure the data came from. Then all five procedure datasets are combined by row into a single master data frame called `squidtime`. This combined object contains every time-to-escape observation for all squids across all procedures.

```r
P1$Trial="P1"
P2$Trial="P2"
P3$Trial="P3"
P4$Trial="P4"
P5$Trial="P5"
```

```r
squidtime=rbind(P1,P2,P3,P4,P5)
```

A second data source, `RAS_FrameSS.ods`, contains the **path distance** (in cm) that each squid swam before reaching the escape hole. These distances were measured by tracking the squid's path from GoPro footage using ImageJ. While `squidtime` tracks *how long* it took to escape, `squiddist` tracks *how far* they swam.

```r
squiddist=readODS::read_ods("RAS_FrameSS.ods",sheet=1)
```

```
## New names:
## * `` -> `...10`
## * `` -> `...11`
```

Merging time and distance data

The distance spreadsheet (`squiddist`) records path length and escape outcome per trial (identified by file number), but does not carry over the trial-level metadata (Tank, Squid, Trial procedure label) from the time dataset. This block merges the two by sorting both by their subject/trial identifiers and binding the relevant columns. It also renames columns in `squiddist` to match the naming conventions used in `squidtime`.

```
squidtime=squidtime[order(squidtime$Code),]
squiddist=squiddist[order(squiddist$`File #`),]
squiddist=cbind(squiddist,squidtime[,c(2,3,6)])

colnames(squiddist)[c(2,6,8)]=c("Code","path_length_cm","Escape..Y.N.")
```

Learning Curves

Assigning run numbers

Within each procedure (trial type), every squid completed 5 repeated runs. This block adds a `Run` column (1–5) to both `squidtime` and `squiddist`, numbering the sequential attempts within each combination of `Tank`, `Squid`, and `Trial`. This allows us later to test whether escape performance improves with practice — i.e., do squids get faster (or more direct) across Runs 1 through 5?

```
squidtime <- squidtime %>%
  arrange(Tank, Squid, Trial, Code) %>%          # make sure they're in order
  group_by(Tank, Squid, Trial) %>%               # each squid in each trial
  mutate(Run = row_number()) %>%                 # 1, 2, 3, 4, 5
  ungroup()


squiddist <- squiddist %>%
  arrange(Tank, Squid, Trial, Code) %>%          # make sure they're in order
  group_by(Tank, Squid, Trial) %>%               # each squid in each trial
  mutate(Run = row_number()) %>%                 # 1, 2, 3, 4, 5
  ungroup()
```

Linear Model Approach

P2 learning curve: time

A simple linear regression of escape time on Run number for the **P2 (White Light Baseline)** procedure. The ANOVA summary tells us whether there is a significant linear change in escape time over the 5 runs. A negative slope would indicate that squids learned to escape faster with practice. Note that the x-axis label "Run" is mislabeled — squiddist data is used but `squidtime` is referenced.

```
P2.lm=lm(Time.Observed..sec.~Run,data=squidtime[squidtime$Trial=="P2",])
summary(aov(P2.lm))
```

```
##             Df  Sum Sq Mean Sq F value Pr(>F)
## Run          1   80328   80328   1.408  0.243
## Residuals   38 2168097   57055
```

P2 learning curve: distance

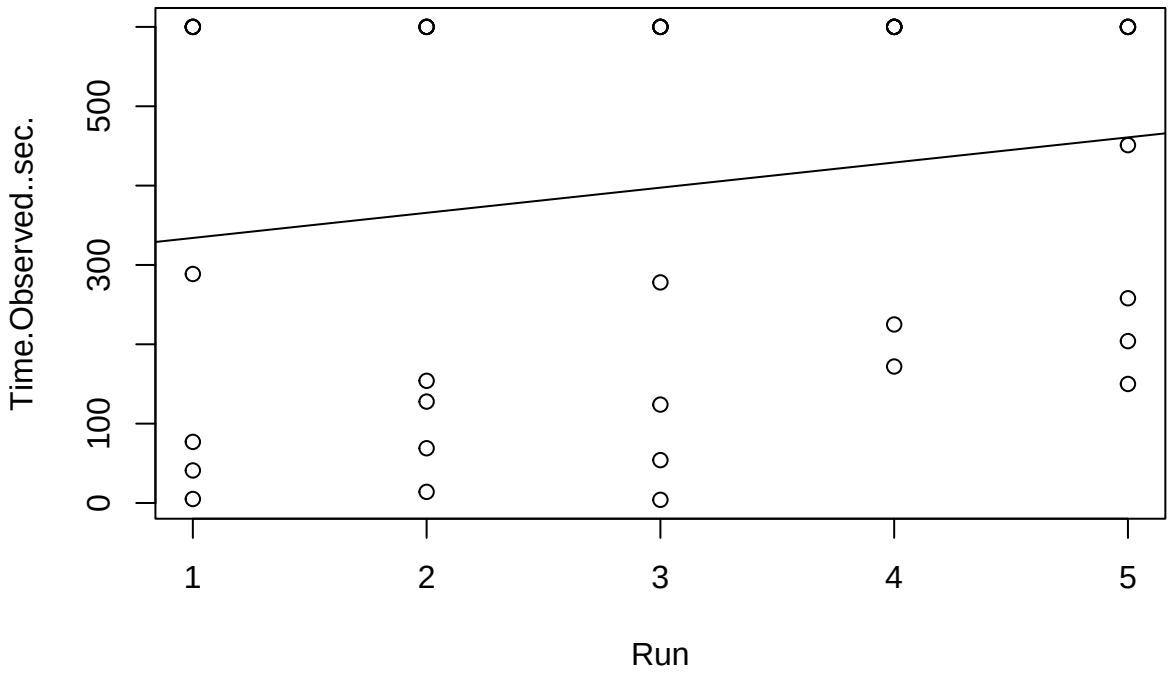
Same approach, but now testing whether the **path length** (distance swum) changed across runs within P2. Again, a significant ANOVA with a negative slope would suggest the squids learned a more direct route.

```r
P2dist.lm=lm(path_length_cm~Run,data=squiddist[squiddist$Trial=="P2",])
summary(aov(P2dist.lm))
```

```
##             Df Sum Sq Mean Sq F value Pr(>F)
## Run          1  20996   20996   4.598 0.0385 *
## Residuals   38 173517    4566
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

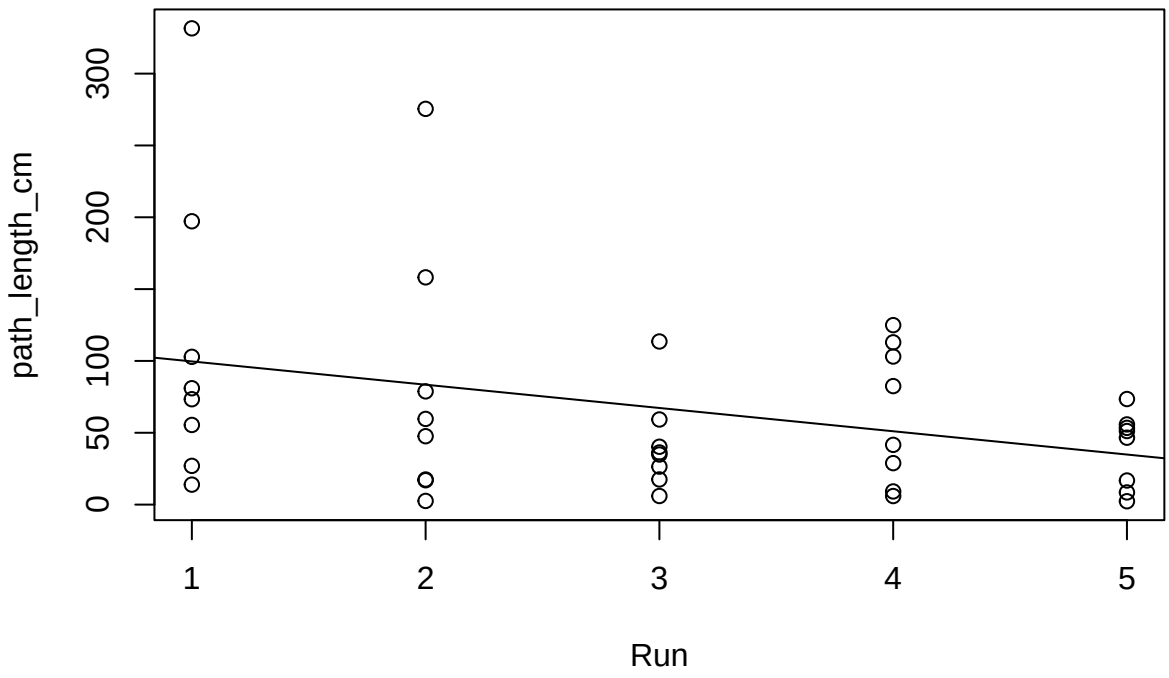
Visual inspection: scatterplot of escape time vs. run number for P2, with the linear regression line overlaid.

```r
plot(Time.Observed..sec.~Run,data=squidtime[squidtime$Trial=="P2",])
abline(P2.lm)
```



Visual inspection: scatterplot of path distance vs. run number for P2, with the regression line.

```r
plot(path_length_cm~Run,data=squiddist[squiddist$Trial=="P2",])
abline(P2dist.lm)
```



Do they escape at a greater rate through P2?

Beyond *how fast* or *how far* they swim, we can ask whether the **probability of escaping at all** increased across runs within P2. This block creates a binary `escape` variable (1 = escaped) and fits a linear model of escape probability vs. Run. Since the outcome is binary, a linear model is not strictly appropriate (logistic regression would be better), but it provides a rough first look at trend.

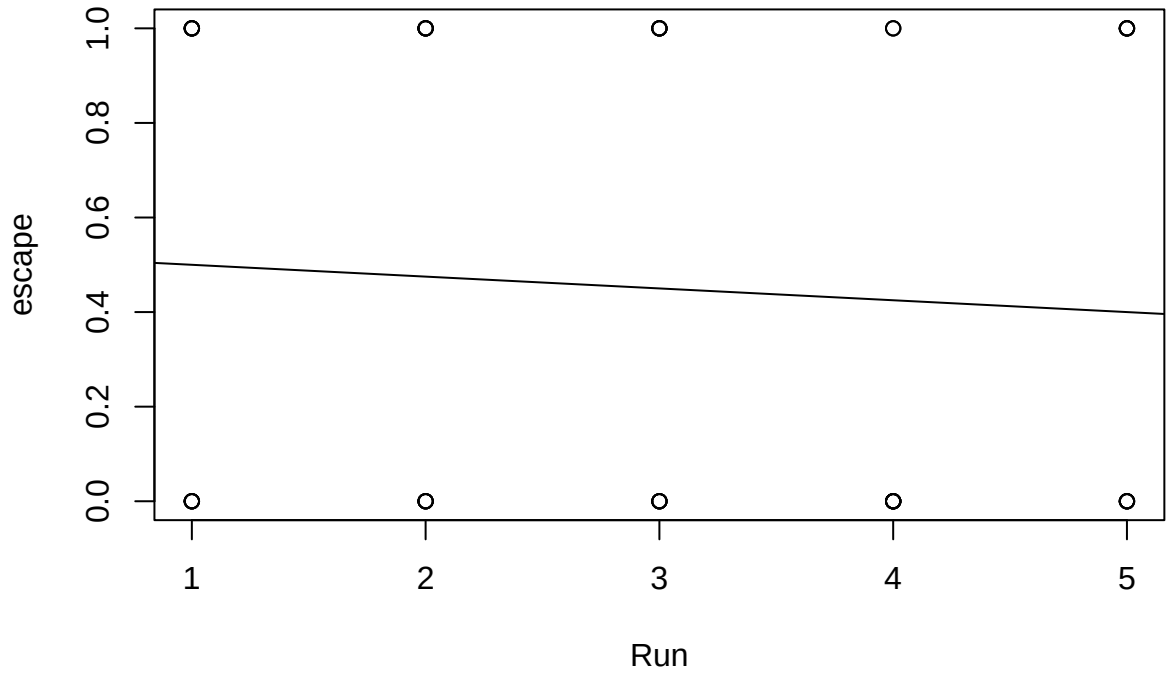
```
squiddist$escape=0
squiddist$escape[squiddist$Escape..Y.N.=="Y"]=1

P2esc.lm=lm(escape~Run,data=squiddist[squiddist$Trial=="P2",])
summary(aov(P2esc.lm))
```

```
##             Df Sum Sq Mean Sq F value Pr(>F)
## Run          1   0.05  0.0500   0.193  0.663
## Residuals   38   9.85  0.2592
```

Scatterplot of escape probability vs. run for P2, with the linear fit line. Same data-indexing issue as above (using `squidtime` filter on `squiddist` data).

```
plot(escape~Run,data=squiddist[squidtime$Trial=="P2",])
abline(P2esc.lm)
```



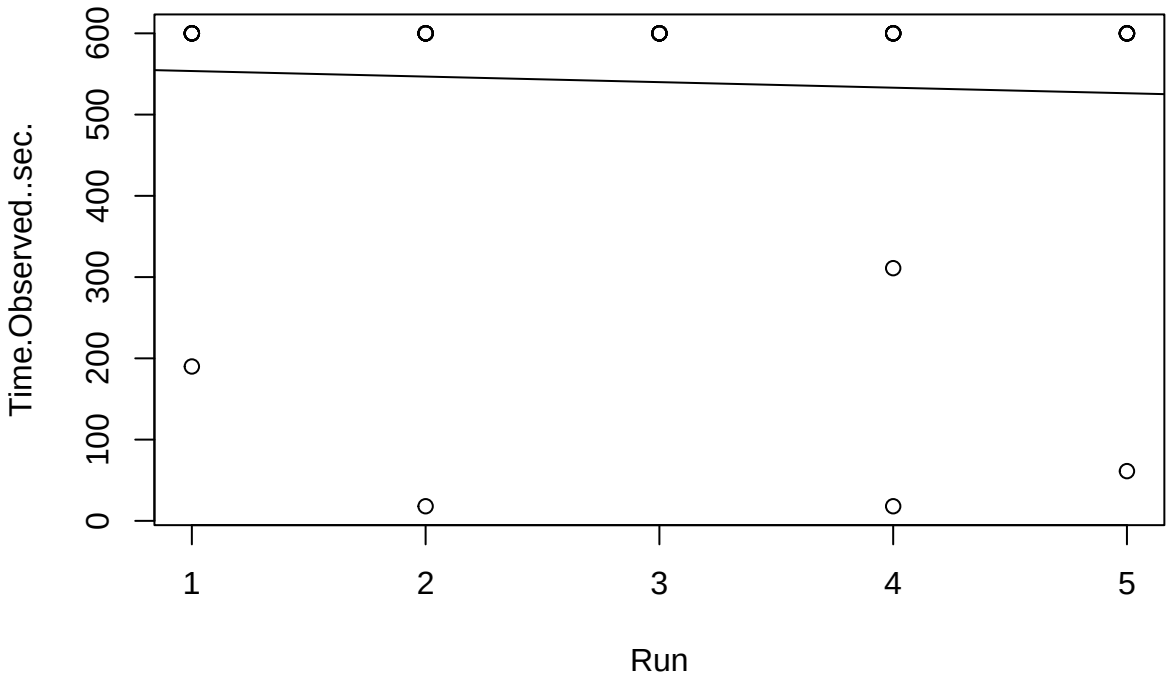
P1 (Red Light Baseline) learning curves

The same learning-curve analysis applied to **P1 (Red Light Baseline)**. Since P1 represents the squids' very first exposure to the novel open-field tank, this tests whether there is an initial learning effect as they become familiar with the arena and locate the escape hole.

```
P1.lm=lm(Time.Observed..sec.~Run,data=squidtime[squidtime$Trial=="P1",])
summary(aov(P1.lm))
```

```
##             Df  Sum Sq Mean Sq F value Pr(>F)
## Run          1    3735    3735   0.132  0.718
## Residuals   38 1071424   28195
```

```
plot(Time.Observed..sec.~Run,data=squidtime[squidtime$Trial=="P1",])
abline(P1.lm)
```

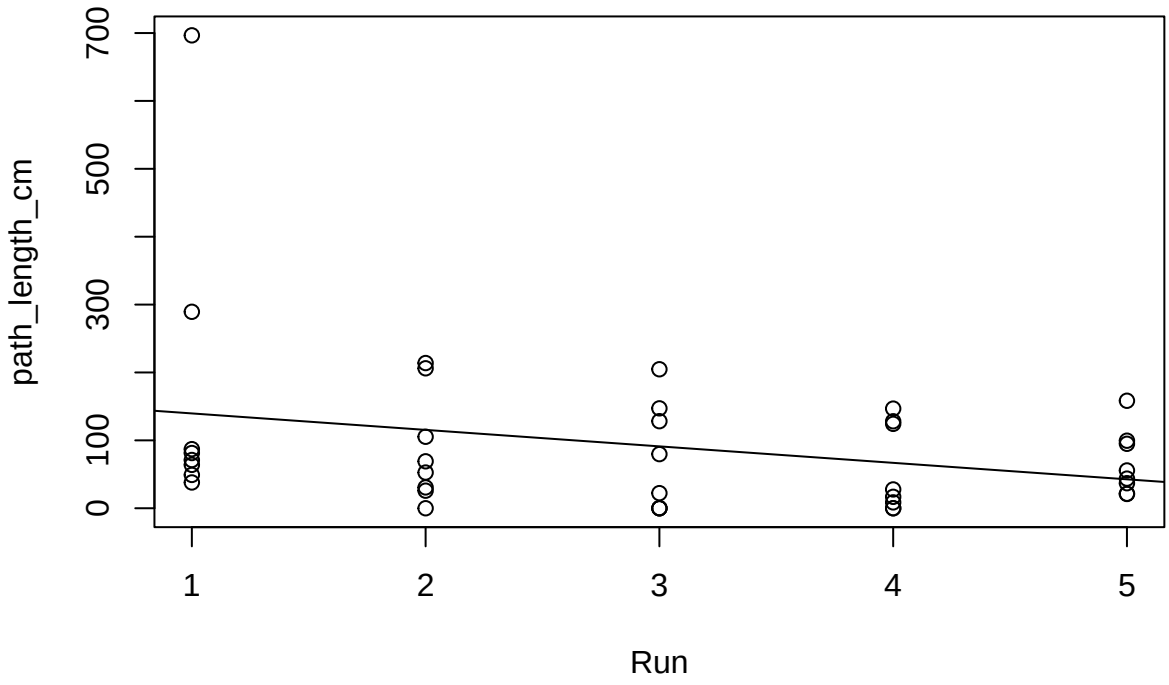


Distance learning curve for P1: do squids also take more direct paths across successive runs in the novel environment?

```
P1dist.lm=lm(path_length_cm~Run,data=squiddist[squiddist$Trial=="P1",])
summary(aov(P1dist.lm))
```

```
##             Df Sum Sq Mean Sq F value Pr(>F)
## Run          1  47109   47109   3.457 0.0708 .
## Residuals   38 517874   13628
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
plot(path_length_cm~Run,data=squiddist[squiddist$Trial=="P1",])
abline(P1dist.lm)
```



GrandRun: learning across all procedures

While `Run` numbers trials *within* each procedure (1–5), `GrandRun` assigns a global trial number across all procedures for each squid (1, 2, 3, …, 25). This allows us to check whether there is improvement across the entire experiment — i.e., do squids become generally better at the escape task over time, beyond within-procedure learning?

```
squidtime <- squidtime %>%
  arrange(Tank, Squid, Code) %>%        # global time order per squid
  group_by(Tank, Squid) %>%             # define an individual squid
  mutate(GrandRun = row_number()) %>%   # 1, 2, 3, ..., 25
  ungroup()
```

Linear regression of escape time on `GrandRun` across all data points. A significant negative slope would indicate that escape efficiency improves cumulatively with experience across the entire experiment.

```
colnames(squidtime)
```

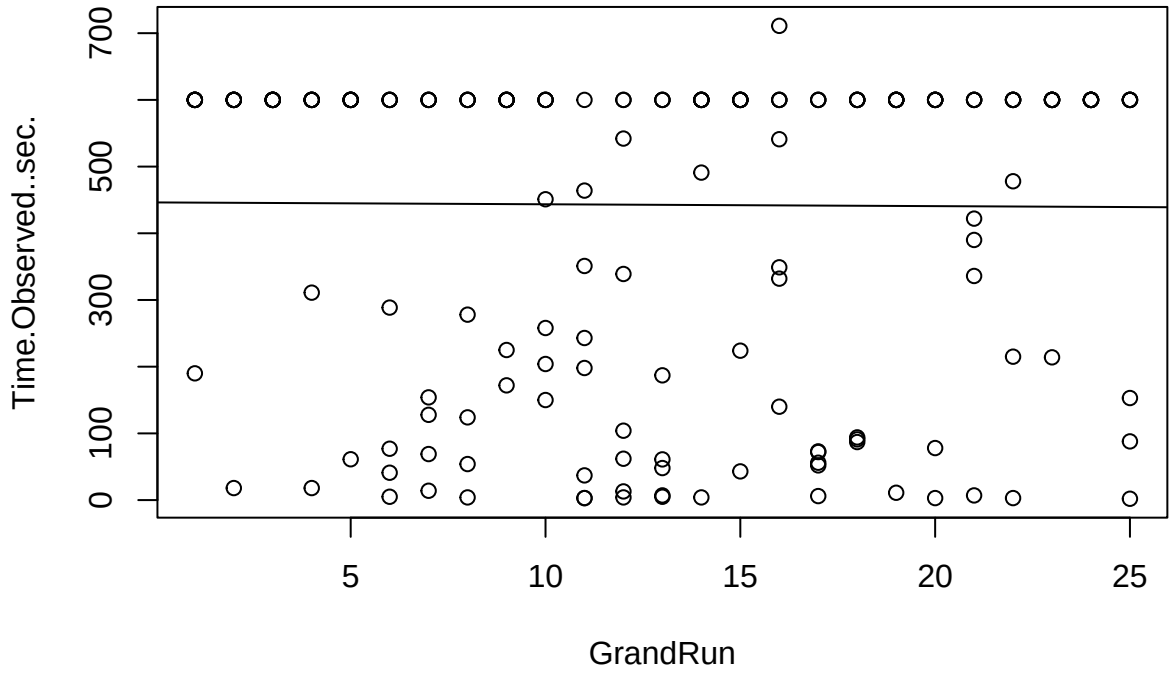
```
## [1] "Code"                "Tank"                "Squid"
## [4] "Escape..Y.N."        "Time.Observed..sec." "Trial"
## [7] "Run"                 "GrandRun"
```

```
grandrun.lm=lm(Time.Observed..sec.~GrandRun,data=squidtime)
summary(aov(grandrun.lm))
```

```
##              Df   Sum Sq Mean Sq F value Pr(>F)
## GrandRun      1      809     809   0.015  0.903
## Residuals   198 10760770   54347
```

Visual check: escape time vs. GrandRun with regression line.

```
plot(Time.Observed..sec.~GrandRun,data=squidtime)
abline(grandrun.lm)
```



Time — all procedures

As a complement to the survival analysis, these simple linear regressions check for within-procedure learning effects (escape time vs. Run) across **all five procedures**. A significant negative slope indicates improvement across the 5 runs within that procedure.

```
colnames(squidtime)
```

```
## [1] "Code"                "Tank"                "Squid"
## [4] "Escape..Y.N."        "Time.Observed..sec." "Trial"
## [7] "Run"                 "GrandRun"
```

```
summary(aov(lm(Time.Observed..sec.~Run,data=squidtime[squidtime$Trial=="P1",])))
```

```
##             Df  Sum Sq Mean Sq F value Pr(>F)
## Run          1    3735    3735   0.132  0.718
## Residuals   38 1071424   28195
```

```
summary(aov(lm(Time.Observed..sec.~Run,data=squidtime[squidtime$Trial=="P2",])))
```

```
##             Df  Sum Sq Mean Sq F value Pr(>F)
## Run          1   80328   80328   1.408  0.243
## Residuals   38 2168097   57055
```

```
summary(aov(lm(Time.Observed..sec.~Run,data=squidtime[squidtime$Trial=="P3",])))
```

```
##             Df  Sum Sq Mean Sq F value Pr(>F)
## Run          1  415729  415729   7.195 0.0108 *
## Residuals   38 2195671   57781
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(lm(Time.Observed..sec.~Run,data=squidtime[squidtime$Trial=="P4",])))
```

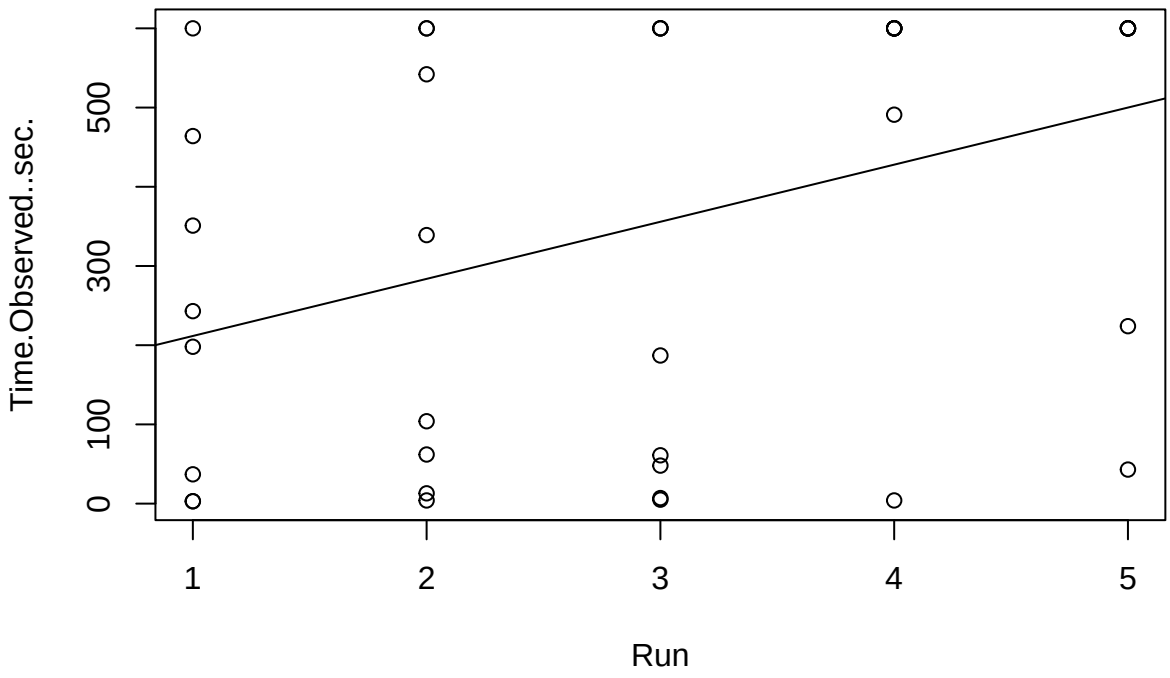
```
##             Df  Sum Sq Mean Sq F value Pr(>F)
## Run          1   39073   39073   0.621  0.436
## Residuals   38 2391213   62927
```

```
summary(aov(lm(Time.Observed..sec.~Run,data=squidtime[squidtime$Trial=="P5",])))
```

```
##             Df  Sum Sq Mean Sq F value Pr(>F)
## Run          1    2880    2880   0.072   0.79
## Residuals   38 1521788   40047
```

Visual check for P3.

```
P3.lm=lm(Time.Observed..sec.~Run,data=squidtime[squidtime$Trial=="P3",])
plot(Time.Observed..sec.~Run,data=squidtime[squidtime$Trial=="P3",])
abline(P3.lm)
```



Distance — all procedures

Same approach for path length across all five procedures.

```
summary(aov(lm(path_length_cm~Run,data=squiddist[squiddist$Trial=="P1",])))
```

```
##             Df Sum Sq Mean Sq F value Pr(>F)
## Run          1  47109   47109   3.457 0.0708 .
## Residuals   38 517874   13628
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(lm(path_length_cm~Run,data=squiddist[squiddist$Trial=="P2",])))
```

```
##             Df Sum Sq Mean Sq F value Pr(>F)
## Run          1  20996   20996   4.598 0.0385 *
## Residuals   38 173517    4566
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(lm(path_length_cm~Run,data=squiddist[squiddist$Trial=="P3",])))
```

```
##             Df Sum Sq Mean Sq F value Pr(>F)
## Run          1   6040    6040    1.47  0.233
## Residuals   38 156199    4110
```

```r
summary(aov(lm(path_length_cm~Run,data=squiddist[squiddist$Trial=="P4",])))
```

```
##             Df Sum Sq Mean Sq F value Pr(>F)
## Run          1  38128   38128   7.199 0.0107 *
## Residuals   38 201260    5296
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
summary(aov(lm(path_length_cm~Run,data=squiddist[squiddist$Trial=="P5",])))
```

```
##             Df Sum Sq Mean Sq F value  Pr(>F)
## Run          1  32253   32253   10.01 0.00306 **
## Residuals   38 122420    3222
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Summary: Linear model learning curves

This table reports the p-values from linear regressions of escape time and path length on Run for each procedure. A significant p-value indicates a linear trend across the 5 runs within that procedure. An asterisk after the p-value indicates significance at the 0.05 level.

```r
format_p <- function(p) {
  p_text <- ifelse(is.na(p), "NA",
                   ifelse(p < 0.001, "<0.001", formatC(p, format = "f", digits = 3)))
  sig <- !is.na(p) & p < 0.05
  result <- cell_spec(p_text, format = "latex", bold = sig)
  ifelse(sig, paste0(result, "*"), result)
}

lm_pvals <- data.frame(
  Procedure = paste0("P", 1:5),
  Time_P = sapply(paste0("P", 1:5), function(tr) {
    m <- lm(Time.Observed..sec. ~ Run, data = squidtime[squidtime$Trial == tr, ])
    summary(aov(m))[[1]]["Run", "Pr(>F)"]
  }),
  Dist_P = sapply(paste0("P", 1:5), function(tr) {
    m <- lm(path_length_cm ~ Run, data = squiddist[squiddist$Trial == tr, ])
    summary(aov(m))[[1]]["Run", "Pr(>F)"]
  })
)

kbl(
  data.frame(
    Procedure = lm_pvals$Procedure,
    Time_P = format_p(lm_pvals$Time_P),
    Dist_P = format_p(lm_pvals$Dist_P)
  ),
  col.names = c("Procedure", "Time P-value", "Dist P-value"),
  format = "latex",
  escape = FALSE,
```

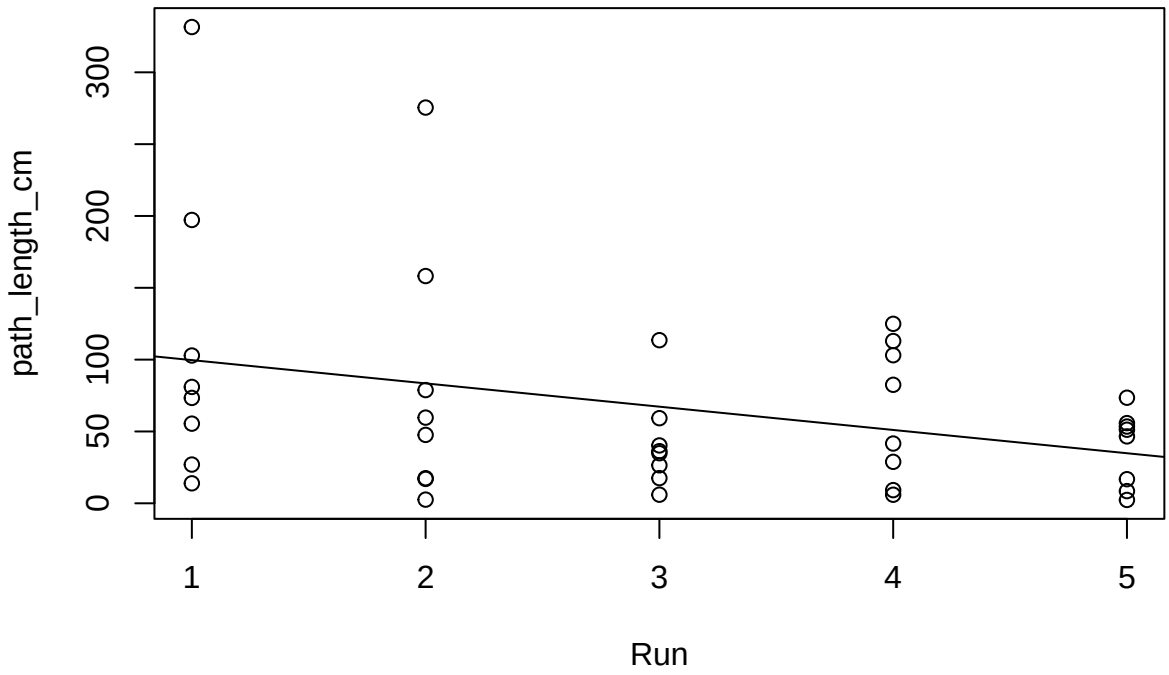
```
  caption = "Linear model learning curves (escape time and path length ~ Run).",
  align = c("l", "c", "c")
) |>
  kable_styling(full_width = FALSE, latex_options = "hold_position") |>
  column_spec(1, bold = TRUE)
```

Table 1: Linear model learning curves (escape time and path length Run).

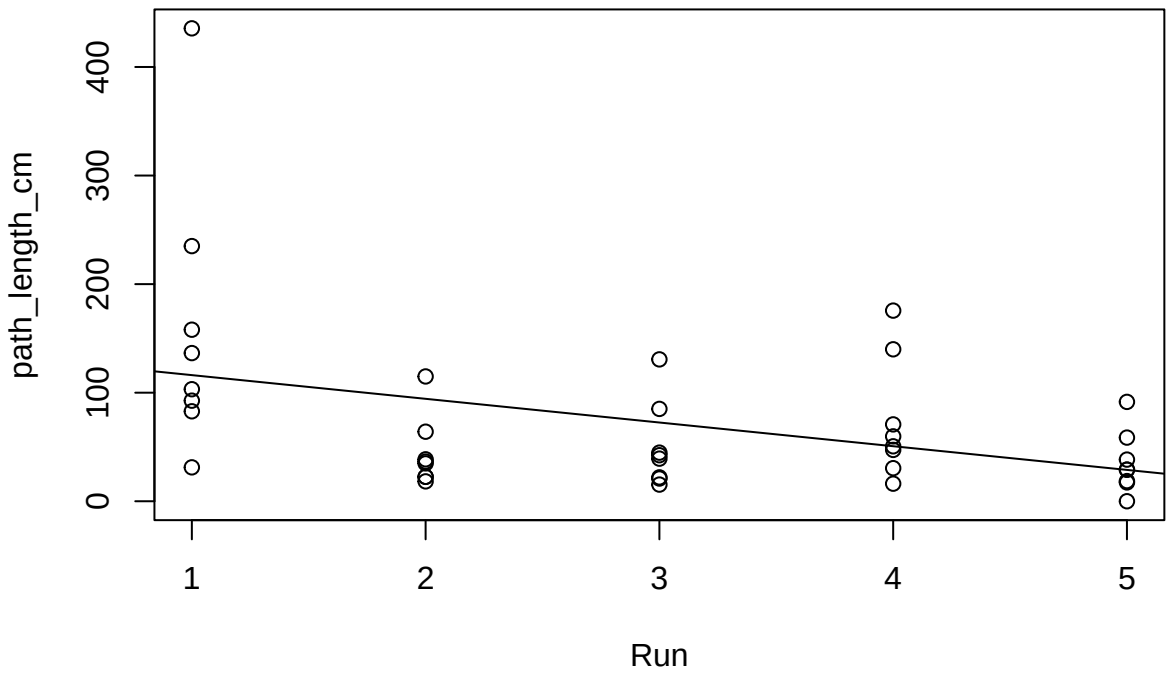
Procedure	Time P-value	Dist P-value
P1	0.718	0.071
P2	0.243	**0.038***
P3	**0.011***	0.233
P4	0.436	**0.011***
P5	0.790	**0.003***

Visual checks for selected procedures.

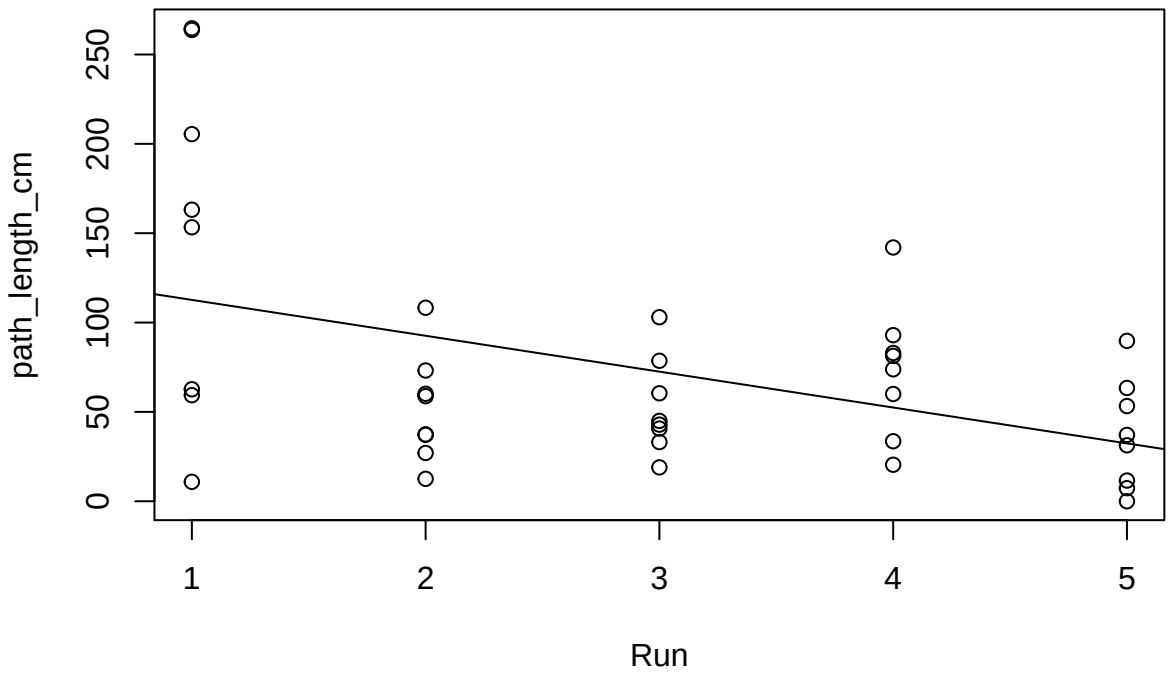
```
P2dist.lm=lm(path_length_cm~Run,data=squiddist[squiddist$Trial=="P2",])
plot(path_length_cm~Run,data=squiddist[squiddist$Trial=="P2",])
abline(P2dist.lm)
```




```
P4dist.lm=lm(path_length_cm~Run,data=squiddist[squiddist$Trial=="P4",])
plot(path_length_cm~Run,data=squiddist[squiddist$Trial=="P4",])
abline(P4dist.lm)
```




```
P5dist.lm=lm(path_length_cm~Run,data=squiddist[squiddist$Trial=="P5",])
plot(path_length_cm~Run,data=squiddist[squiddist$Trial=="P5",])
abline(P5dist.lm)
```



Quick peek at the structure of the combined dataset.

```
head(squidtime)
```

```
## # A tibble: 6 x 8
##    Code  Tank Squid Escape..Y.N. Time.Observed..sec. Trial   Run GrandRun
##   <dbl> <dbl> <dbl> <chr>                      <dbl> <chr> <int>    <int>
## 1 12002     1     1 no                           600 P1        1        1
## 2 12003     1     1 no                           600 P1        2        2
## 3 12004     1     1 no                           600 P1        3        3
## 4 12005     1     1 no                           600 P1        4        4
## 5 12006     1     1 no                           600 P1        5        5
## 6 12042     1     1 no                           600 P2        1        6
```

Survival Analysis Approach

Within-trial Cox models (P2–P5)

For each procedure individually (excluding P1), fit a Cox model of escape time against Run with a random intercept for Tank/Squid. This provides procedure-specific estimates of the **within-procedure learning rate** — the per-run change in escape hazard within that procedure. A positive coefficient for Run means escape hazard increases (escape time decreases) across successive runs.

```r
fit_within_trial <- function(trial_name) {
  dat <- subset(squidtime, Trial == trial_name)
  coxme(Surv(Time.Observed..sec., Escape..Y.N.=="yes") ~ Run + (1 | Tank/Squid),
        data = dat)
}

mP2 <- fit_within_trial("P2")
mP3 <- fit_within_trial("P3")
mP4 <- fit_within_trial("P4")
mP5 <- fit_within_trial("P5")

summary(mP2); summary(mP3); summary(mP4); summary(mP5)
```

```
## Mixed effects coxme model
##  Formula: Surv(Time.Observed..sec., Escape..Y.N. == "yes") ~ Run + (1 |      Tank/Squid)
##     Data: dat
##
##   events, n = 18, 40
##
## Random effects:
##        group    variable        sd  variance
## 1 Tank/Squid (Intercept) 0.4866210 0.2368000
## 2       Tank (Intercept) 0.6172085 0.3809464
##                   Chisq   df      p   AIC   BIC
## Integrated loglik  3.05 3.00 0.3840 -2.95 -5.62
##  Penalized loglik 11.44 4.36 0.0285  2.72 -1.16
##
## Fixed effects:
##        coef exp(coef) se(coef)     z     p
## Run -0.2064    0.8135   0.1705 -1.21 0.226
```

```
## Mixed effects coxme model
##  Formula: Surv(Time.Observed..sec., Escape..Y.N. == "yes") ~ Run + (1 |      Tank/Squid)
##     Data: dat
##
##   events, n = 22, 40
##
## Random effects:
##        group    variable         sd   variance
## 1 Tank/Squid (Intercept) 0.01996196 0.00039848
## 2       Tank (Intercept) 1.06407019 1.13224536
##                   Chisq   df         p   AIC   BIC
## Integrated loglik 17.91 3.00 4.591e-04 11.91  8.64
##  Penalized loglik 25.82 3.51 1.972e-05 18.79 14.96
##
## Fixed effects:
##        coef exp(coef) se(coef)    z       p
## Run -0.5224    0.5931   0.1632 -3.2 0.00137
```

```
## Mixed effects coxme model
##  Formula: Surv(Time.Observed..sec., Escape..Y.N. == "yes") ~ Run + (1 |      Tank/Squid)
##     Data: dat
##
```

```
##   events, n = 16, 40
##
## Random effects:
##        group    variable         sd     variance
## 1 Tank/Squid (Intercept) 0.01999191 0.0003996766
## 2       Tank (Intercept) 0.56909424 0.3238682577
##                    Chisq   df       p   AIC   BIC
## Integrated loglik  3.21 3.00 0.36088 -2.79 -5.11
##  Penalized loglik  7.10 2.62 0.05095  1.87 -0.16
##
## Fixed effects:
##        coef exp(coef) se(coef)     z    p
## Run -0.2696    0.7637   0.1826 -1.48 0.14
```

```
## Mixed effects coxme model
##  Formula: Surv(Time.Observed..sec., Escape..Y.N. == "yes") ~ Run + (1 |      Tank/Squid)
##     Data: dat
##
##   events, n = 11, 40
##
## Random effects:
##        group    variable         sd     variance
## 1 Tank/Squid (Intercept) 0.45753949 0.2093423805
## 2       Tank (Intercept) 0.01993868 0.0003975511
##                    Chisq   df      p   AIC   BIC
## Integrated loglik  1.41 3.00 0.7031 -4.59 -5.78
##  Penalized loglik  4.72 2.55 0.1455 -0.39 -1.40
##
## Fixed effects:
##        coef exp(coef) se(coef)     z     p
## Run -0.2364    0.7894   0.2289 -1.03 0.302
```

Figure: Relative escape hazard by run — time

This is the key visualization for the **learning curve** question. For each procedure, a within-trial Cox model is fit (escape time ~ Run, with random intercept for Tank/Squid), and the **log relative hazard** is plotted for Runs 1–5 relative to Run 1 (reference).

The y-axis shows the log -transformed hazard ratio. Values above 0 mean the escape hazard is higher (squid escapes faster) relative to Run 1; values below 0 mean lower hazard (slower escape). A positive slope across runs indicates learning — the squid is increasingly likely to escape sooner with each successive run within that procedure.

The helper function `get_logrelhaz_by_run` fits the model and computes the linear predictor and 95% confidence intervals for each run. Standard errors scale with the distance from the reference run.

```
# --- settings ---
trials_to_plot <- c("P1","P2","P3","P4","P5")
runs <- 1:5
ref_run <- 1
titles=c("P1: Red Light Baseline",
         "P2: White Light Baseline",
         "P3: Opposite Start Location", #Path Integration
         "P4: Rearrange Landmarks", #Piloting
```

```
         "P5: Remove Beacon") #Beaconing

shade_col <- adjustcolor("#6cc0a8", alpha.f = 0.25)
line_col  <- "#6cc0a8"

# Ensure ordering
squidtime$Trial <- factor(squidtime$Trial, levels = trials_to_plot)

# Helper: fit within-trial model and return log-relative hazard vs Run=1 (+ CI)
get_logrelhaz_by_run <- function(dat_trial, runs = 1:5, ref_run = 1) {

  m <- coxme(Surv(Time.Observed..sec., Escape..Y.N. == "yes") ~ Run + (1 | Tank/Squid),
             data = dat_trial)

  b <- as.numeric(fixef(m)["Run"])
  V <- as.matrix(vcov(m))    # 1x1 (fixed effect Run only)

  d_run <- runs - ref_run

  lp <- b * d_run                          # log relative hazard
  se_lp <- sqrt(V[1,1]) * abs(d_run)       # SE scales with |Run-ref|

out <- data.frame(
  Run = runs,
  lp = lp / log(2),
  lo = (lp - 1.96 * se_lp) / log(2),
  hi = (lp + 1.96 * se_lp) / log(2)
)

  list(model = m, pred = out)
}

# --- compute predictions for each trial ---
pred_list <- setNames(vector("list", length(trials_to_plot)), trials_to_plot)
for (tr in trials_to_plot) {
  dat_tr <- subset(squidtime, Trial == tr)
  pred_list[[tr]] <- get_logrelhaz_by_run(dat_tr, runs = runs, ref_run = ref_run)
}

# --- common y-limits across panels (recommended) ---
all_lo <- unlist(lapply(pred_list, function(z) z$pred$lo))
all_hi <- unlist(lapply(pred_list, function(z) z$pred$hi))
yl <- range(c(all_lo, all_hi), finite = TRUE)

# --- 5-panel layout ---
png("LogRelHaz_byRun_5panels.png", width = 12, height = 8, units = "in", res = 300)
par(mfrow = c(2,3), mar = c(4,4,3,1), oma = c(0,0,2,0), bg="white", mgp=c(1.7,0.7,0))
iter=1
for (tr in trials_to_plot) {
  d <- pred_list[[tr]]$pred

  plot(d$Run, d$lp, type = "n",
       xlim = c(1,5), ylim = yl,
```

```
       xlab = "Run",
       ylab = expression(Log[2] * " Relative Escape Hazard (vs Run 1)"),
       cex.lab=1.3,
       main = titles[iter],
       axes = FALSE)

  axis(1, at = 1:5)
  axis(2, las = 1)
  box()
  text(1,2.1,"More Likely to Escape →", srt=90,cex=0.7)
  text(1,-2.1,"← Less Likely to Escape", srt=90, cex=0.7)
  # Reference line at 0 (same hazard as Run 1)
  abline(h = 0, lty = 3)

  # Shaded CI band
  polygon(c(d$Run, rev(d$Run)),
          c(d$lo,  rev(d$hi)),
          col = shade_col, border = NA)

  # Estimate line + points
  lines(d$Run, d$lp, lwd = 2, col = line_col)
  points(d$Run, d$lp, pch = 16, cex = 0.9, col = line_col)
  iter=iter+1
}

# blank 6th panel
plot.new()

#mtext("Within-trial change in escape hazard across runs (each panel relative to Run 1)",
#      outer = TRUE, cex = 1.1)

dev.off()
```

```
## pdf
##   2
```

Figure: Relative escape hazard by run — distance

Same as the time-based figure above, but using **path distance** as the survival outcome. This tells us whether the **efficiency of the escape route** (not just speed) improves across runs within each procedure.

```
library(coxme)
library(survival)

# --- settings ---
trials_to_plot <- c("P1","P2","P3","P4","P5")
runs <- 1:5
ref_run <- 1
titles=c("P1: Red Light Baseline",
         "P2: White Light Baseline",
         "P3: Opposite Start Location", #Path Integration
         "P4: Rearrange Landmarks", #Piloting
```

```
         "P5: Remove Beacon") #Beaconing

shade_col <- adjustcolor("#6cc0a8", alpha.f = 0.25)
line_col  <- "#6cc0a8"

# Ensure ordering
squiddist$Trial <- factor(squiddist$Trial, levels = trials_to_plot)

# Helper: fit within-trial model and return log-relative hazard vs Run=1 (+ CI)
get_logrelhaz_by_run <- function(dat_trial, runs = 1:5, ref_run = 1) {

  m <- coxme(Surv(path_length_cm, Escape..Y.N. == "Y") ~ Run + (1 | Tank/Squid),
             data = dat_trial)

  b <- as.numeric(fixef(m)["Run"])
  V <- as.matrix(vcov(m))    # 1x1 (fixed effect Run only)

  d_run <- runs - ref_run

  lp <- b * d_run                          # log relative hazard
  se_lp <- sqrt(V[1,1]) * abs(d_run)       # SE scales with |Run-ref|

out <- data.frame(
  Run = runs,
  lp = lp / log(2),
  lo = (lp - 1.96 * se_lp) / log(2),
  hi = (lp + 1.96 * se_lp) / log(2)
)

  list(model = m, pred = out)
}

# --- compute predictions for each trial ---
pred_list <- setNames(vector("list", length(trials_to_plot)), trials_to_plot)
for (tr in trials_to_plot) {
  dat_tr <- subset(squiddist, Trial == tr)
  pred_list[[tr]] <- get_logrelhaz_by_run(dat_tr, runs = runs, ref_run = ref_run)
}

# --- common y-limits across panels (recommended) ---
all_lo <- unlist(lapply(pred_list, function(z) z$pred$lo))
all_hi <- unlist(lapply(pred_list, function(z) z$pred$hi))
yl <- range(c(all_lo, all_hi), finite = TRUE)

# --- 5-panel layout ---
png("LogRelHaz_Dist_byRun_5panels.png", width = 12, height = 8, units = "in", res = 300)
par(mfrow = c(2,3), mar = c(4,4,3,1), oma = c(0,0,2,0), bg="white", mgp=c(1.7,0.7,0))
iter=1
for (tr in trials_to_plot) {
  d <- pred_list[[tr]]$pred

  plot(d$Run, d$lp, type = "n",
       xlim = c(1,5), ylim = yl,
```

```
       xlab = "Run",
       ylab = expression(Log[2] * " Relative Escape Hazard (vs Run 1)"),
       cex.lab=1.3,
       main = titles[iter],
       axes = FALSE)

  axis(1, at = 1:5)
  axis(2, las = 1)
  box()
  text(1,2.1,"More Likely to Escape →", srt=90,cex=0.7)
  text(1,-2.1,"← Less Likely to Escape", srt=90, cex=0.7)
  # Reference line at 0 (same hazard as Run 1)
  abline(h = 0, lty = 3)

  # Shaded CI band
  polygon(c(d$Run, rev(d$Run)),
          c(d$lo,  rev(d$hi)),
          col = shade_col, border = NA)

  # Estimate line + points
  lines(d$Run, d$lp, lwd = 2, col = line_col)
  points(d$Run, d$lp, pch = 16, cex = 0.9, col = line_col)
  iter=iter+1
}

# blank 6th panel
plot.new()

#mtext("Within-trial change in escape hazard across runs (each panel relative to Run 1)",
#      outer = TRUE, cex = 1.1)

dev.off()
```

```
## pdf
##   2
```

Summary: Within-procedure learning curves

This table reports the p-values for the Run effect from the Cox mixed-effects models fit above for each procedure. Time models used `mP2`–`mP5`; distance models used the models returned by `get_logrelhaz_by_run()`. A significant p-value indicates that escape time or distance changed across runs within that procedure. An asterisk after the p-value indicates significance at the 0.05 level.

```
cox_within <- data.frame(
  Procedure = paste0("P", 1:5),
  Time_P = c(NA, sapply(list(mP2, mP3, mP4, mP5), function(m) {
    tryCatch(summary(m)$coefficients["Run", "p"], error = function(e) NA)
  })),
  Dist_P = sapply(paste0("P", 1:5), function(tr) {
    tryCatch(summary(pred_list[[tr]]$model)$coefficients["Run", "p"], error = function(e) NA)
  })
)
```

```r
kbl(
  data.frame(
    Procedure = cox_within$Procedure,
    Time_P = format_p(cox_within$Time_P),
    Dist_P = format_p(cox_within$Dist_P)
  ),
  col.names = c("Procedure", "Time P-value", "Dist P-value"),
  format = "latex",
  escape = FALSE,
  caption = "Within-procedure learning curves from Cox mixed-effects models.",
  align = c("l", "c", "c")
) |>
  kable_styling(full_width = FALSE, latex_options = "hold_position") |>
  column_spec(1, bold = TRUE)
```

Table 2: Within-procedure learning curves from Cox mixed-effects models.

Procedure	Time P-value	Dist P-value
P1	NA	0.567
P2	0.226	0.851
P3	**0.001***	0.086
P4	0.140	0.765
P5	0.302	0.528

Does GrandRun (global experience) improve fit?

This tests whether a **global learning effect** across the entire experiment (GrandRun) improves the model compared to a null model with only random effects. In other words, do squids become generally better at the escape task regardless of which procedure they're in?

```r
m0 <- coxme(Surv(Time.Observed..sec., Escape..Y.N.=="yes") ~ (1 | Tank/Squid),
            data = squidtime)

m1 <- coxme(Surv(Time.Observed..sec., Escape..Y.N.=="yes") ~ GrandRun + (1 | Tank/Squid),
            data = squidtime)

anova(m0, m1)   # tests whether adding GrandRun improves fit
```

```
## Analysis of Deviance Table
##  Cox model: response is  Surv(Time.Observed..sec., Escape..Y.N. == "yes")
##  Model 1: ~(1 | Tank/Squid)
##  Model 2: ~GrandRun + (1 | Tank/Squid)
##    loglik  Chisq Df P(>|Chi|)
## 1 -355.81
## 2 -355.73 0.1649  1    0.6847
```

Summary of the model with GrandRun as a fixed effect. A negative coefficient for GrandRun means that for each successive trial (regardless of procedure), the hazard of escape increases — i.e., the squid escapes faster.

```r
summary(m1)
```

```
## Mixed effects coxme model
##  Formula: Surv(Time.Observed..sec., Escape..Y.N. == "yes") ~ GrandRun +      (1 | Tank/Squid)
##     Data: squidtime
##
##   events, n = 72, 200
##
## Random effects:
##        group    variable        sd  variance
## 1 Tank/Squid (Intercept) 0.5563070 0.3094774
## 2       Tank (Intercept) 0.4214975 0.1776601
##                    Chisq   df         p   AIC   BIC
## Integrated loglik 12.47 3.00 0.0059360  6.47 -0.36
##  Penalized loglik 28.25 6.34 0.0001136 15.57  1.14
##
## Fixed effects:
##              coef exp(coef) se(coef)    z     p
## GrandRun 0.006343  1.006363 0.015611 0.41 0.685
```

Comparison of Procedures

Survival analysis (Cox proportional hazards models) is the core analytical framework of this study. Unlike linear regression, which models the mean outcome, survival analysis models the **time until an event occurs** — here, the "event" is the squid escaping the tank. This approach has two key advantages:

1. It naturally handles **censored data**: trials where the squid did not escape within the 10-minute cutoff. These observations are not discarded — they contribute information that the escape time is *at least* 10 minutes.
2. The **Cox mixed-effects model** (`coxme`) can include random effects to account for the hierarchical structure of the data: multiple runs nested within individual squids, nested within tanks.

The central question: when a spatial cue is altered or removed (P3–P5), do escape times (or distances) increase compared to the baseline procedure (P2)? A positive coefficient for the test procedure would indicate reduced escape efficiency, suggesting the squid relied on that cue.

P3 (Opposite Start Location) vs. P2 (White Light Baseline) — Time

P3 tests path integration: the starting position is shifted 90° from P2's position. If the squid located the hole using internal self-motion cues (path integration), changing the start position should not affect performance — they would simply compute a new vector. If instead they relied on external cues aligned with the tank or room, performance might differ.

Data are subset to P2 and P3 only, and `Trial` is releveled so P2 serves as the reference category.

```r
p2p3 <- squidtime %>%
  filter(Trial %in% c("P2", "P3")) %>%
  droplevels()
```

```r
p2p3$Trial <- factor(p2p3$Trial)   # drops ordering
p2p3$Trial <- relevel(p2p3$Trial, ref = "P2")
```

The escape event variable is created from the `Escape..Y.N.` column. Note that in `squidtime` (the Excel source), escape is coded as lowercase `"yes"` / `"no"`.

```r
p2p3$event <- ifelse(p2p3$Escape..Y.N. == "yes", 1, 0)
```

A mixed-effects Cox model with `Trial` (P2 vs. P3) as a fixed effect and a random intercept for `Tank/Squid` (squid nested within tank). The model tests whether the hazard of escape differs between P3 and P2. A hazard ratio < 1 (negative coefficient) means squids in P3 took longer to escape; a hazard ratio > 1 means they escaped faster.

```r
m_p2p3 <- coxme(
  Surv(Time.Observed..sec., event) ~ Trial + (1 | Tank/Squid),
  data = p2p3
)
summary(m_p2p3)
```

```
## Mixed effects coxme model
##  Formula: Surv(Time.Observed..sec., event) ~ Trial + (1 | Tank/Squid)
##     Data: p2p3
##
##   events, n = 40, 80
##
## Random effects:
##        group    variable        sd  variance
## 1 Tank/Squid (Intercept) 0.3883035 0.1507796
## 2       Tank (Intercept) 0.7603689 0.5781609
##                    Chisq  df         p   AIC  BIC
## Integrated loglik 11.27 3.0 0.0103730  5.27 0.20
##  Penalized loglik 23.33 5.2 0.0003513 12.92 4.14
##
## Fixed effects:
##           coef exp(coef) se(coef)    z     p
## TrialP3 0.4306    1.5383   0.3229 1.33 0.182
```

P3 vs. P2 — Distance

Same comparison, but now using **path length (cm)** as the survival time variable. Instead of "time to escape," the outcome is "distance traveled to escape." This tests whether squids took a less direct route when the start position was moved, even if total time was not significantly affected.

```r
p2p3dist <- squiddist %>%
  filter(Trial %in% c("P2", "P3")) %>%
  droplevels()
```

```r
p2p3dist$Trial <- factor(p2p3dist$Trial)   # drops ordering
p2p3dist$Trial <- relevel(p2p3dist$Trial, ref = "P2")
```

Note: in the distance data (`squiddist`), the escape column uses uppercase `"Y"` / `"N"`, unlike the lowercase `"yes"` / `"no"` in `squidtime`. This is a data-source inconsistency.

```
p2p3dist$event <- ifelse(p2p3dist$Escape..Y.N. == "Y", 1, 0)
```

```
m_p2p3dist <- coxme(
  Surv(path_length_cm, event) ~ Trial + (1 | Tank/Squid),
  data = p2p3dist
)
summary(m_p2p3dist)
```

```
## Mixed effects coxme model
##  Formula: Surv(path_length_cm, event) ~ Trial + (1 | Tank/Squid)
##     Data: p2p3dist
##
##   events, n = 40, 80
##
## Random effects:
##        group    variable         sd     variance
## 1 Tank/Squid (Intercept) 0.01986302 0.0003945397
## 2       Tank (Intercept) 0.55248344 0.3052379521
##                    Chisq   df       p   AIC   BIC
## Integrated loglik  4.24 3.00 0.23669 -1.76 -6.83
##  Penalized loglik 10.41 3.21 0.01845  3.99 -1.42
##
## Fixed effects:
##           coef exp(coef) se(coef)    z    p
## TrialP3 0.1834    1.2013   0.3226 0.57 0.57
```

P1 (Red Light Baseline) vs. P2 (White Light Baseline) — Time

This comparison was not part of the original proposal's plan. P1 and P2 are both baseline conditions — the only difference is the lighting (red vs. white light). Squids were first tested under red light (P1, which mimics their natural dim/nighttime environment), then under white light (P2). This analysis asks whether the lighting change itself affected escape performance.

Note: this model includes `Run` as an additional fixed effect (alongside `Trial`), which is not done in the other between-procedure comparisons. This accounts for within-procedure learning when comparing P1 and P2.

```
p1p2 <- squidtime %>%
  filter(Trial %in% c("P1", "P2")) %>%
  droplevels()
```

```
p1p2$Trial <- factor(p1p2$Trial)   # drops ordering
p1p2$Trial <- relevel(p1p2$Trial, ref = "P1")
```

```
p1p2$event <- ifelse(p1p2$Escape..Y.N. == "yes", 1, 0)
```

```
m_p1p2 <- coxme(
  Surv(Time.Observed..sec., event) ~ Trial + Run + (1 | Tank/Squid),
  data = p1p2
)
summary(m_p1p2)
```

```
## Mixed effects coxme model
##  Formula: Surv(Time.Observed..sec., event) ~ Trial + Run + (1 | Tank/Squid)
##     Data: p1p2
##
##   events, n = 23, 80
##
## Random effects:
##        group    variable        sd  variance
## 1 Tank/Squid (Intercept) 0.5625277 0.3164375
## 2       Tank (Intercept) 0.3466213 0.1201463
##                    Chisq   df        p   AIC BIC
## Integrated loglik 12.94 4.00 0.011566  4.94 0.4
##  Penalized loglik 21.82 5.65 0.000987 10.52 4.1
##
## Fixed effects:
##            coef exp(coef) se(coef)     z       p
## TrialP2  1.5692    4.8030   0.5123  3.06 0.00219
## Run     -0.1042    0.9010   0.1482 -0.70 0.48207
```

Kaplan–Meier plot: P1 vs. P2 — survival (proportion not escaped)

Before fitting the Cox model, we can visualize the raw escape-time distributions using Kaplan–Meier curves. This plot shows the **proportion of squids that have NOT yet escaped** (survival) as a function of time. P1 and P2 are distinguished by line type (both black). The vertical dashed line at 600 s marks the 10-minute trial cutoff — squids that never escaped are censored here.

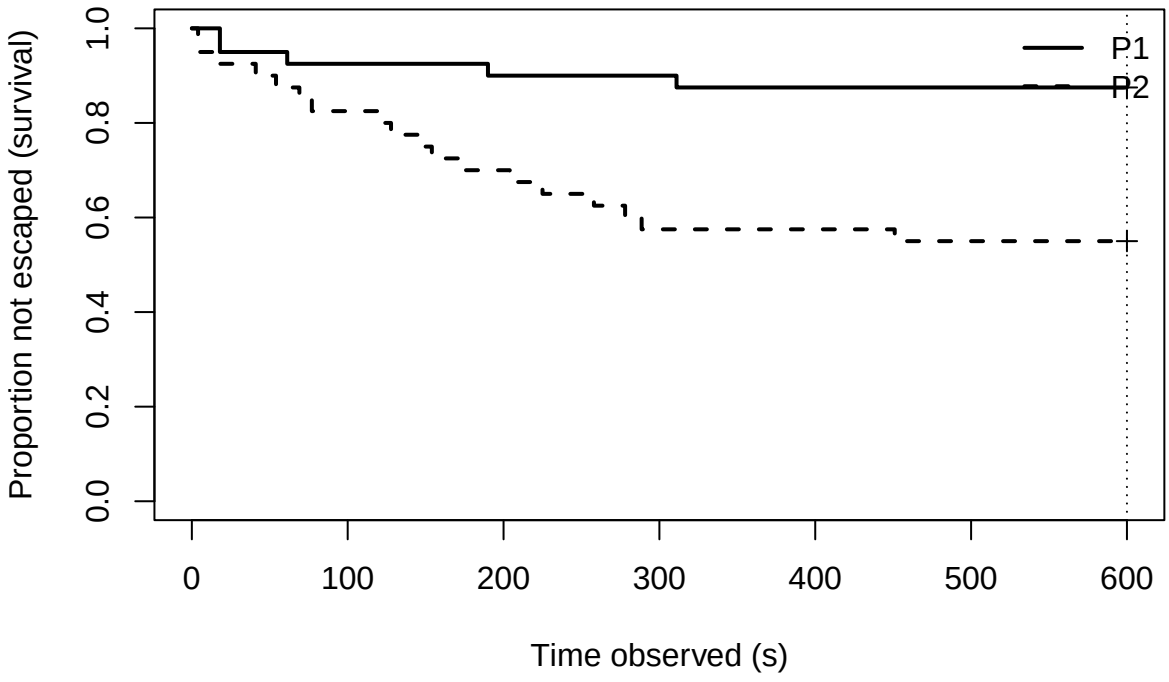
```r
## assuming you already created p1p2 and p1p2$event
## (keeping your existing code style)

# Kaplan-Meier curves by Trial (descriptive)
km_p1p2 <- survfit(Surv(Time.Observed..sec., event) ~ Trial, data = p1p2)

# Plot in base R
plot(km_p1p2,
     xlab = "Time observed (s)",
     ylab = "Proportion not escaped (survival)",
     xlim = c(0, 600),
     mark.time = TRUE,      # marks censoring times
     lwd = 2,
     lty = c(1, 2),
     col = c(1, 1))         # both black; distinguish via lty

legend("topright",
       legend = levels(p1p2$Trial),
       lty = c(1, 2),
       lwd = 2,
       bty = "n")

abline(v = 600, lty = 3)    # optional: show censoring cutoff time
```



Publication figure: P1 vs. P2 — cumulative proportion escaped

This block generates a polished figure saved as `P1P2_Survival.png`. Unlike the quick plot above, this uses `fun = "event"` to show the **cumulative proportion escaped** (i.e., 1 - survival) with confidence intervals. P1 is shown in green (`#6cc0a8`) and P2 in purple (`#5A4B62`).

```
p1p2$Trial <- factor(p1p2$Trial, levels = c("P1", "P2"))
# Kaplan-Meier curves by Trial (descriptive)
km_p1p2 <- survfit(Surv(Time.Observed..sec., event) ~ Trial, data = p1p2)

png("P1P2_Survival.png",width=9,height=9,units = "in",res = 300)
par(bg="white")
plot(km_p1p2,
     fun = "event",conf.int=TRUE,       # plots 1 - S(t)
     xlab = "Time observed (s)",
     ylab = "Cumulative Proportion escaped",
     xlim = c(0, 600),
     mark.time = TRUE,
     lwd = 2,
     lty = c(1, 2),
     col = c("#6cc0a8", "#5A4B62"),
     axes=F)

legend("topleft",
       legend = levels(p1p2$Trial),
```

```
       lty = c(1, 2),
       lwd = 2,
       col = c("#6cc0a8", "#5A4B62"),
       bty = "n")

abline(v = 600, lty = 3)
axis(1)
axis(2)
dev.off()
```

```
## pdf
##   2
```

Custom CI figure: P1 vs. P2 — cumulative escaped with shaded confidence bands

This block builds a fully custom KM figure from scratch (rather than using `plot.survfit`), because the default plot does not cleanly display confidence intervals on the event scale (cumulative proportion escaped). The `step_xy()` helper function converts survival step functions into coordinates suitable for drawing polygons and lines.

The figure saved as `P1P2_Survival_CI.png` shows: - Cumulative escape curves (event = 1 - survival) - Shaded 95% confidence bands (semi-transparent green/purple) - The 600 s censoring cutoff as a vertical dashed line

```
library(survival)

p1p2$Trial <- factor(p1p2$Trial, levels = c("P1", "P2"))
km_p1p2 <- survfit(Surv(Time.Observed..sec., event) ~ Trial, data = p1p2)

# helper: convert (t, y) into step coordinates (for type="s" shapes)
step_xy <- function(t, y, x0 = 0, y0 = 0) {
  t <- c(x0, t)
  y <- c(y0, y)
  xs <- rep(t, each = 2)[-1]
  ys <- rep(y, each = 2)
  ys <- ys[-length(ys)]
  list(x = xs, y = ys)
}

png("P1P2_Survival_CI.png", width = 12, height = 8, units = "in", res = 300)
par(bg = "white")

plot(0, 0, type = "n",
     xlim = c(0, 600),
     ylim = c(0, 0.6),
     xlab = "Time observed (s)",
     ylab = "Cumulative proportion escaped",
     axes = FALSE)
axis(1)
axis(2, las = 1)
box()
abline(v = 600, lty = 3)
```

```r
line_cols <- c("#6cc0a8", "#5A4B62")
fill_cols <- c(adjustcolor("#6cc0a8", alpha.f = 0.25),
               adjustcolor("#5A4B62", alpha.f = 0.25))

ncurves <- length(km_p1p2$strata)
idx <- 1

for (i in seq_len(ncurves)) {

  n_i <- km_p1p2$strata[i]
  t <- km_p1p2$time[idx:(idx + n_i - 1)]

  surv  <- km_p1p2$surv[idx:(idx + n_i - 1)]
  lower <- km_p1p2$lower[idx:(idx + n_i - 1)]
  upper <- km_p1p2$upper[idx:(idx + n_i - 1)]

  # extend to 600 so the last segment is flat to the censoring limit
  if (max(t) < 600) {
    t <- c(t, 600)
    surv  <- c(surv,  tail(surv,  1))
    lower <- c(lower, tail(lower, 1))
    upper <- c(upper, tail(upper, 1))
  }

  # convert to event scale: F(t) = 1 - S(t)
  event <- 1 - surv
  event_lower <- 1 - upper   # note: swap for correct CI on event scale
  event_upper <- 1 - lower

  # step coords for CI band
  lo <- step_xy(t, event_lower)
  up <- step_xy(t, event_upper)

  polygon(c(lo$x, rev(up$x)),
          c(lo$y, rev(up$y)),
          col = fill_cols[i],
          border = NA)

  # step coords for estimate line
  est <- step_xy(t, event)
  lines(est$x, est$y, lwd = 2, lty = i, col = line_cols[i])

  idx <- idx + n_i
}

legend("topleft",
       legend = c("P1: Red light", "P2: White light"),
       lty = 1:2,
       lwd = 2,
       col = line_cols,
       bty = "n")

dev.off()
```

```
## pdf
##   2
```

P1 vs. P2 — Distance

The same P1-vs-P2 comparison, but using **path length** instead of time. Did squids take more or less direct routes under white light (P2) compared to red light (P1)?

```
p1p2dist <- squiddist %>%
  filter(Trial %in% c("P1", "P2")) %>%
  droplevels()
```

```
p1p2dist$Trial <- factor(p1p2dist$Trial)   # drops ordering
p1p2dist$Trial <- relevel(p1p2dist$Trial, ref = "P1")
```

```
p1p2dist$event <- ifelse(p1p2dist$Escape..Y.N. == "Y", 1, 0)
```

```
m_p1p2dist <- coxme(
  Surv(path_length_cm, event) ~ Trial + Run + (1 | Tank/Squid),
  data = p1p2dist
)
summary(m_p1p2dist)
```

```
## Mixed effects coxme model
##  Formula: Surv(path_length_cm, event) ~ Trial + Run + (1 | Tank/Squid)
##     Data: p1p2dist
##
##   events, n = 23, 80
##
## Random effects:
##        group    variable         sd     variance
## 1 Tank/Squid (Intercept) 0.01993233 0.0003972978
## 2       Tank (Intercept) 0.38273965 0.1464896401
##                    Chisq   df        p  AIC   BIC
## Integrated loglik 10.71 4.00 0.029964 2.71 -1.83
##  Penalized loglik 13.85 3.35 0.004368 7.16  3.36
##
## Fixed effects:
##            coef exp(coef) se(coef)    z       p
## TrialP2 1.48928   4.43388  0.51169 2.91 0.00361
## Run     0.08741   1.09134  0.14761 0.59 0.55375
```

Kaplan–Meier plot: P1 vs. P2 — distance survival

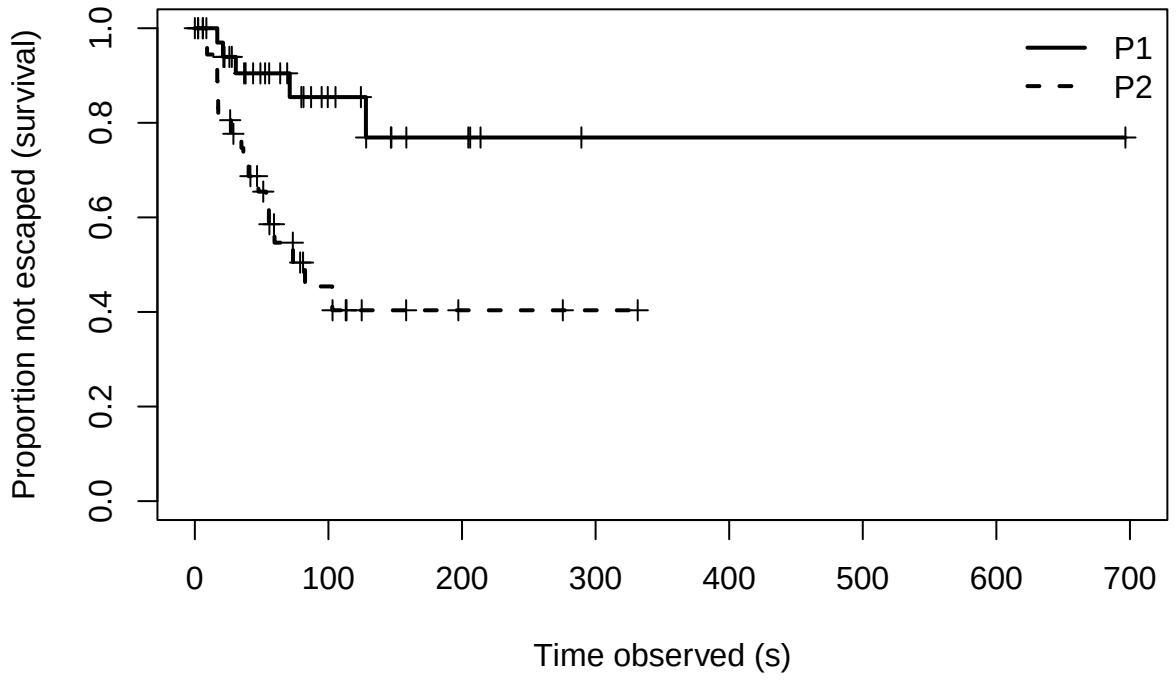
Exploratory KM plot using path length as the time variable. Note the x-axis label mistakenly says "Time observed (s)" when it should read "Distance traveled (cm)" — the `plot.survfit` call uses the same x-axis label as the time-based plots rather than an appropriate label for distance.

```
## assuming you already created p1p2 and p1p2$event
## (keeping your existing code style)
```

```r
# Kaplan-Meier curves by Trial (descriptive)
km_p1p2dist <- survfit(Surv(path_length_cm, event) ~ Trial, data = p1p2dist)

# Plot in base R
plot(km_p1p2dist,
     xlab = "Time observed (s)",
     ylab = "Proportion not escaped (survival)",
     xlim = c(0, round(max(p1p2dist$path_length_cm),digits=-2)),
     mark.time = TRUE,      # marks censoring times
     lwd = 2,
     lty = c(1, 2),
     col = c(1, 1))         # both black; distinguish via lty

legend("topright",
       legend = levels(p1p2dist$Trial),
       lty = c(1, 2),
       lwd = 2,
       bty = "n")
```



Custom CI figure: P1 vs. P2 — distance

Same approach as the time-based CI figure, but now using path length as the x-axis. The x-axis limit is rounded from the maximum observed distance. The x-axis label still says "Time observed (s)" rather than "Distance traveled (cm)" — a copy-paste issue from the time-based version.

```r
library(survival)

p1p2dist$Trial <- factor(p1p2dist$Trial, levels = c("P1", "P2"))
km_p1p2dist <- survfit(Surv(path_length_cm, event) ~ Trial, data = p1p2dist)

# helper: convert (t, y) into step coordinates (for type="s" shapes)
step_xy <- function(t, y, x0 = 0, y0 = 0) {
  t <- c(x0, t)
  y <- c(y0, y)
  xs <- rep(t, each = 2)[-1]
  ys <- rep(y, each = 2)
  ys <- ys[-length(ys)]
  list(x = xs, y = ys)
}

png("P1P2dist_Survival_CI.png", width = 12, height = 8, units = "in", res = 300)
par(bg = "white")

plot(0, 0, type = "n",
     xlim = c(0, round(max(p1p2dist$path_length_cm),digits=-2)),
     ylim = c(0, 1),
     xlab = "Distance traveled (cm)",
     ylab = "Cumulative proportion escaped",
     axes = FALSE)
axis(1)
axis(2, las = 1)
box()

line_cols <- c("#6cc0a8", "#5A4B62")
fill_cols <- c(adjustcolor("#6cc0a8", alpha.f = 0.25),
               adjustcolor("#5A4B62", alpha.f = 0.25))

ncurves <- length(km_p1p2dist$strata)
idx <- 1

for (i in seq_len(ncurves)) {

  n_i <- km_p1p2dist$strata[i]
  t <- km_p1p2dist$time[idx:(idx + n_i - 1)]

  surv  <- km_p1p2dist$surv[idx:(idx + n_i - 1)]
  lower <- km_p1p2dist$lower[idx:(idx + n_i - 1)]
  upper <- km_p1p2dist$upper[idx:(idx + n_i - 1)]

  # convert to event scale: F(t) = 1 - S(t)
  event <- 1 - surv
  event_lower <- 1 - upper   # note: swap for correct CI on event scale
  event_upper <- 1 - lower

  # step coords for CI band
  lo <- step_xy(t, event_lower)
  up <- step_xy(t, event_upper)
```

```
  polygon(c(lo$x, rev(up$x)),
          c(lo$y, rev(up$y)),
          col = fill_cols[i],
          border = NA)

  # step coords for estimate line
  est <- step_xy(t, event)
  lines(est$x, est$y, lwd = 2, lty = i, col = line_cols[i])

  idx <- idx + n_i
}

legend("topleft",
       legend = c("P1: Red light", "P2: White light"),
       lty = 1:2,
       lwd = 2,
       col = line_cols,
       bty = "n")

dev.off()
```

```
## pdf
##   2
```

P2 vs. P4 (Rearrange Landmarks) — Time

P4 tests piloting navigation: the internal tank objects (rocks, plants) are rearranged while the escape hole location remains the same. If squids use a cognitive map of the landmark layout, this rearrangement should impair performance (longer escape times). The model compares P4 to the P2 baseline.

```
p2p4 <- squidtime %>%
  filter(Trial %in% c("P2", "P4")) %>%
  droplevels()
```

```
p2p4$Trial <- factor(p2p4$Trial)   # drops ordering
p2p4$Trial <- relevel(p2p4$Trial, ref = "P2")
```

```
p2p4$event <- ifelse(p2p4$Escape..Y.N. == "yes", 1, 0)
```

```
m_p2p4 <- coxme(
  Surv(Time.Observed..sec., event) ~ Trial + (1 | Tank/Squid),
  data = p2p4
)
summary(m_p2p4)
```

```
## Mixed effects coxme model
##  Formula: Surv(Time.Observed..sec., event) ~ Trial + (1 | Tank/Squid)
##     Data: p2p4
##
##   events, n = 34, 80
##
```

```
## Random effects:
##        group    variable        sd  variance
## 1 Tank/Squid (Intercept) 0.5605612 0.3142288
## 2       Tank (Intercept) 0.5219147 0.2723949
##                   Chisq   df        p   AIC   BIC
## Integrated loglik  5.94 3.00 0.114470 -0.06 -4.64
##  Penalized loglik 17.83 5.47 0.004543  6.90 -1.45
##
## Fixed effects:
##            coef exp(coef) se(coef)     z     p
## TrialP4 -0.1961    0.8220   0.3510 -0.56 0.576
```

P2 vs. P4 — Distance

The same piloting test, but with path distance as the outcome.

```
p2p4dist <- squiddist %>%
  filter(Trial %in% c("P2", "P4")) %>%
  droplevels()
```

```
p2p4dist$Trial <- factor(p2p4dist$Trial)   # drops ordering
p2p4dist$Trial <- relevel(p2p4dist$Trial, ref = "P2")
```

```
p2p4dist$event <- ifelse(p2p4dist$Escape..Y.N. == "Y", 1, 0)
```

```
m_p2p4dist <- coxme(
  Surv(path_length_cm, event) ~ Trial + (1 | Tank/Squid),
  data = p2p4dist
)
summary(m_p2p4dist)
```

```
## Mixed effects coxme model
##  Formula: Surv(path_length_cm, event) ~ Trial + (1 | Tank/Squid)
##     Data: p2p4dist
##
##   events, n = 34, 80
##
## Random effects:
##        group    variable         sd     variance
## 1 Tank/Squid (Intercept) 0.01999946 0.0003999782
## 2       Tank (Intercept) 0.62798825 0.3943692379
##                   Chisq   df       p   AIC  BIC
## Integrated loglik  4.68 3.00 0.19714 -1.32 -5.9
##  Penalized loglik 11.04 3.25 0.01431  4.55 -0.4
##
## Fixed effects:
##            coef exp(coef) se(coef)     z     p
## TrialP4 -0.1586    0.8533   0.3478 -0.46 0.648
```

P2 vs. P5 (Remove Beacon) — Time

P5 tests beaconing navigation: the floating marker (ping pong ball) positioned over the escape hole is removed. If squids were using this marker as a beacon to locate the hole, its removal should increase escape times.

```
p2p5 <- squidtime %>%
  filter(Trial %in% c("P2", "P5")) %>%
  droplevels()
```

```
p2p5$Trial <- factor(p2p5$Trial)   # drops ordering
p2p5$Trial <- relevel(p2p5$Trial, ref = "P2")
```

```
p2p5$event <- ifelse(p2p5$Escape..Y.N. == "yes", 1, 0)
```

```
m_p2p5 <- coxme(
  Surv(Time.Observed..sec., event) ~ Trial + (1 | Tank/Squid),
  data = p2p5
)
summary(m_p2p5)
```

```
## Mixed effects coxme model
##  Formula: Surv(Time.Observed..sec., event) ~ Trial + (1 | Tank/Squid)
##     Data: p2p5
##
##   events, n = 29, 80
##
## Random effects:
##        group    variable         sd    variance
## 1 Tank/Squid (Intercept) 0.70205027 0.492874577
## 2       Tank (Intercept) 0.02000265 0.000400106
##                   Chisq   df        p  AIC   BIC
## Integrated loglik  7.23 3.00 0.064901 1.23 -2.87
##  Penalized loglik 18.37 5.37 0.003367 7.64  0.30
##
## Fixed effects:
##            coef exp(coef) se(coef)    z      p
## TrialP5 -0.7321    0.4809   0.3863 -1.9 0.0581
```

P2 vs. P5 — Distance

The beaconing test with path distance as the outcome.

```
p2p5dist <- squiddist %>%
  filter(Trial %in% c("P2", "P5")) %>%
  droplevels()
```

```
p2p5dist$Trial <- factor(p2p5dist$Trial)   # drops ordering
p2p5dist$Trial <- relevel(p2p5dist$Trial, ref = "P2")
```

```r
p2p5dist$event <- ifelse(p2p5dist$Escape..Y.N. == "Y", 1, 0)
```

```r
m_p2p5dist <- coxme(
  Surv(path_length_cm, event) ~ Trial + (1 | Tank/Squid),
  data = p2p5dist
)
summary(m_p2p5dist)
```

```
## Mixed effects coxme model
##  Formula: Surv(path_length_cm, event) ~ Trial + (1 | Tank/Squid)
##     Data: p2p5dist
##
##   events, n = 29, 80
##
## Random effects:
##        group    variable        sd   variance
## 1 Tank/Squid (Intercept) 0.2126853 0.04523502
## 2       Tank (Intercept) 0.5948775 0.35387922
##                   Chisq   df       p   AIC   BIC
## Integrated loglik  5.51 3.00 0.13807 -0.49 -4.59
##  Penalized loglik 12.37 3.63 0.01094  5.12  0.16
##
## Fixed effects:
##            coef exp(coef) se(coef)     z    p
## TrialP5 -0.6043    0.5464   0.3887 -1.55 0.12
```

Exponentiating the model coefficient gives the hazard ratio for P5 relative to P2 on the distance outcome.

```r
exp(m_p2p5dist$coefficients)
```

```
##   TrialP5
## 0.5464357
```

Summary: Between-procedure comparisons

This table reports the p-values for the Trial effect from the Cox mixed-effects models comparing pairs of procedures. A significant p-value indicates that escape time or distance differed between the two procedures. An asterisk after the p-value indicates significance at the 0.05 level.

```r
cox_btwn <- data.frame(
  Comparison = c("P1 vs P2", "P3 vs P2", "P4 vs P2", "P5 vs P2"),
  Time_P = sapply(list(m_p1p2, m_p2p3, m_p2p4, m_p2p5), function(m) {
    tryCatch({
      coefs <- summary(m)$coefficients
      trial_row <- grep("^Trial", rownames(coefs))
      if(length(trial_row) == 0) NA else coefs[trial_row, "p"]
    }, error = function(e) NA)
  }),
  Dist_P = sapply(list(m_p1p2dist, m_p2p3dist, m_p2p4dist, m_p2p5dist), function(m) {
    tryCatch({
      coefs <- summary(m)$coefficients
```

```
      trial_row <- grep("^Trial", rownames(coefs))
      if(length(trial_row) == 0) NA else coefs[trial_row, "p"]
    }, error = function(e) NA)
  })
)

kbl(
  data.frame(
    Comparison = cox_btwn$Comparison,
    Time_P = format_p(cox_btwn$Time_P),
    Dist_P = format_p(cox_btwn$Dist_P)
  ),
  col.names = c("Comparison", "Time P-value", "Dist P-value"),
  format = "latex",
  escape = FALSE,
  caption = "Between-procedure comparisons from Cox mixed-effects models.",
  align = c("l", "c", "c")
) |>
  kable_styling(full_width = FALSE, latex_options = "hold_position") |>
  column_spec(1, bold = TRUE)
```

Table 3: Between-procedure comparisons from Cox mixed-effects models.

Comparison	Time P-value	Dist P-value
P1 vs P2	**0.002***	**0.004***
P3 vs P2	0.182	0.570
P4 vs P2	0.576	0.648
P5 vs P2	0.058	0.120

Grand Cox model: Trial × Run interaction

This mixed-effects Cox model includes **Trial**, **Run**, and their interaction as fixed effects, excluding P1 (red light baseline). It asks a comprehensive question: does the learning trajectory (slope of Run) differ across procedures? A significant Trial × Run interaction would indicate that within-procedure learning rates depend on which navigational cues are available.

```
grand.m <- tryCatch(
  coxme(
    Surv(Time.Observed..sec., Escape..Y.N. == "yes") ~
      Trial * Run +
      (1 | Tank/Squid),
    data = squidtime[squidtime$Trial!="P1",]
  ),
  error = function(e) {
    message("Grand Cox model failed: ", conditionMessage(e))
    NULL
  }
)
```

```
## Grand Cox model failed: No starting estimate was successful
```

```r
if(!is.null(grand.m)) summary(grand.m)
```

Figure: Time survival curves — P2 vs. P3/P4/P5

This 3-panel figure visualizes the **cumulative proportion escaped** (event = 1 - survival) for each navigation-manipulation procedure (P3, P4, P5) compared against the P2 white-light baseline. Each panel shows two Kaplan–Meier curves with shaded 95% confidence bands.

The `plot_pair_km()` function: 1. Fits a Kaplan–Meier estimator of survival stratified by Trial 2. Converts to the event scale (proportion escaped) 3. Truncates curves at `tmax = 600` s (the 10-minute trial cutoff) and extends flat to the limit for clean polygons 4. Draws shaded confidence bands and step-function estimate lines

The `step_xy()` helper converts a step function (time, value) into alternating x-y coordinates suitable for `polygon()` and `lines()`.

```r
library(survival)

# Helper: convert (t, y) into step coordinates (horizontal then vertical)
step_xy <- function(t, y, x0 = 0, y0 = 0) {
  t <- c(x0, t)
  y <- c(y0, y)
  xs <- rep(t, each = 2)[-1]
  ys <- rep(y, each = 2)
  ys <- ys[-length(ys)]
  list(x = xs, y = ys)
}

# Plot one panel: KM curves (proportion escaped) for a pair of trials, with shaded CI
plot_pair_km <- function(dat, trial_levels, main_title,
                         tmax = 600,
                         line_cols = c("#6cc0a8", "#5A4B62"),
                         ltys = c(1, 2),
                         alpha = 0.20) {

  dat$Trial <- factor(dat$Trial, levels = trial_levels)

  fit <- survfit(Surv(Time.Observed..sec., Escape..Y.N.=="yes") ~ Trial, data = dat)

  fill_cols <- adjustcolor(line_cols, alpha.f = alpha)

  # Empty canvas
  plot(0, 0, type = "n",
       xlim = c(0, tmax),
       ylim = c(0, 1),
       xlab = "Time observed (s)",
       ylab = "Cumulative proportion escaped",
       cex.lab=1.3,
       axes = FALSE)
  axis(1)
  axis(2, las = 1)
  box()
  abline(v = tmax, lty = 3)
```

```r
# Draw each stratum (2 curves)
ncurves <- length(fit$strata)
idx <- 1

for (i in seq_len(ncurves)) {
  n_i <- fit$strata[i]

  t <- fit$time[idx:(idx + n_i - 1)]
  surv  <- fit$surv[idx:(idx + n_i - 1)]
  lower <- fit$lower[idx:(idx + n_i - 1)]
  upper <- fit$upper[idx:(idx + n_i - 1)]

  # Advance idx now so we don't forget on early 'next'
  idx <- idx + n_i

  if (length(t) == 0) next

  # ---- FIX: truncate everything to tmax for clean polygons ----
  keep <- t <= tmax
  t     <- t[keep]
  surv  <- surv[keep]
  lower <- lower[keep]
  upper <- upper[keep]

  if (length(t) == 0) next

  # Ensure curve ends exactly at tmax (flat extension)
  if (max(t) < tmax) {
    t <- c(t, tmax)
    surv  <- c(surv,  tail(surv,  1))
    lower <- c(lower, tail(lower, 1))
    upper <- c(upper, tail(upper, 1))
  }

  # event scale: proportion escaped = 1 - S(t)
  event    <- 1 - surv
  event_lo <- 1 - upper   # swapped for correct CI on event scale
  event_hi <- 1 - lower

  # Step coords for CI band
  lo <- step_xy(t, event_lo)
  hi <- step_xy(t, event_hi)

  polygon(c(lo$x, rev(hi$x)),
          c(lo$y, rev(hi$y)),
          col = fill_cols[i], border = NA)

  # Step coords for estimate
  est <- step_xy(t, event)
  lines(est$x, est$y, lwd = 2, lty = ltys[i], col = line_cols[i])
}
```

```
}

ltys <- c(1, 2)
line_cols <- c("#6cc0a8", "#5A4B62")

# ---- Build the 3-panel figure: P2 vs P3, P2 vs P4, P2 vs P5 ----
png("P2_vs_P3P4P5_3panel.png", width = 12, height = 8, units = "in", res = 300)
par(bg="white", mfrow = c(1,3), mar = c(4,4,3,1), mgp = c(2,0.7,0))

# P2 vs P3
dat23 <- subset(squidtime, Trial %in% c("P2","P3"))
plot_pair_km(dat23, trial_levels = c("P2","P3"), main_title = "P2 vs P3", tmax = 600)
  legend("topleft",
         legend = c("P2: White Light Baseline","P3: Opposite Start Location"),
         lty = ltys, lwd = 2, col = line_cols,
         bty = "n", cex = 0.9)

# P2 vs P4 (handles times > 600 cleanly via truncation)
dat24 <- subset(squidtime, Trial %in% c("P2","P4"))
plot_pair_km(dat24, trial_levels = c("P2","P4"), main_title = "P2 vs P4", tmax = 600)
  legend("topleft",
         legend = c("P2: White Light Baseline","P4: Rearrange Landmarks"),
         lty = ltys, lwd = 2, col = line_cols,
         bty = "n", cex = 0.9)

# P2 vs P5
dat25 <- subset(squidtime, Trial %in% c("P2","P5"))
plot_pair_km(dat25, trial_levels = c("P2","P5"), main_title = "P2 vs P5", tmax = 600)
  legend("topleft",
         legend = c("P2: White Light Baseline","P5: Remove Beacon"),
         lty = ltys, lwd = 2, col = line_cols,
         bty = "n", cex = 0.9)

dev.off()
```

```
## pdf
##   2
```

Figure: Distance survival curves — P2 vs. P3/P4/P5

Same 3-panel layout as the time figure, but now using **path distance** as the survival variable. The x-axis label is corrected to "Distance traveled (cm)" (unlike the P1P2 distance comparison, which retained the time label by mistake). The y-axis shows the cumulative proportion of squids that have escaped by a given distance traveled.

A key difference from the time version: the function here does NOT truncate at 600 cm, because distance has no natural cutoff analogous to the 10-minute time limit. The x-axis limit is determined by the maximum observed distance. Confidence bands are only drawn where CI bounds are valid (non-NA), which avoids gaps at the right tail where few observations remain.

```
library(survival)

# Helper: convert (t, y) into step coordinates (horizontal then vertical)
```

```r
step_xy <- function(t, y, x0 = 0, y0 = 0) {
  t <- c(x0, t)
  y <- c(y0, y)
  xs <- rep(t, each = 2)[-1]
  ys <- rep(y, each = 2)
  ys <- ys[-length(ys)]
  list(x = xs, y = ys)
}

# Plot one panel: KM curves (proportion escaped) for a pair of trials, with shaded CI
plot_pair_km <- function(dat, trial_levels, main_title,
                         line_cols = c("#6cc0a8", "#5A4B62"),
                         ltys = c(1, 2),
                         alpha = 0.20) {

  dat$Trial <- factor(dat$Trial, levels = trial_levels)

  fit <- survfit(Surv(path_length_cm, Escape..Y.N.=="Y") ~ Trial, data = dat)

  fill_cols <- adjustcolor(line_cols, alpha.f = alpha)

  # Set x-axis limit from data
  xmax <- max(fit$time)

  # Empty canvas
  plot(0, 0, type = "n",
       xlim = c(0, xmax),
       ylim = c(0, 1),
       xlab = "Distance traveled (cm)",
       ylab = "Cumulative proportion escaped",
       main = main_title,
       cex.lab = 1.3,
       axes = FALSE)
  axis(1)
  axis(2, las = 1)
  box()

  # Draw each stratum (2 curves)
  ncurves <- length(fit$strata)
  idx <- 1

  for (i in seq_len(ncurves)) {
    n_i <- fit$strata[i]

    t <- fit$time[idx:(idx + n_i - 1)]
    surv  <- fit$surv[idx:(idx + n_i - 1)]
    lower <- fit$lower[idx:(idx + n_i - 1)]
    upper <- fit$upper[idx:(idx + n_i - 1)]

    idx <- idx + n_i

    if (length(t) == 0) next
```

```
    # event scale: proportion escaped = 1 - S(t)
    event    <- 1 - surv
    event_lo <- 1 - upper
    event_hi <- 1 - lower

    # Only draw CI band and line where CI bounds are valid (not NA)
    valid <- !is.na(event_lo) & !is.na(event_hi)

    if (sum(valid) > 1) {
      lo <- step_xy(t[valid], event_lo[valid])
      hi <- step_xy(t[valid], event_hi[valid])

      polygon(c(lo$x, rev(hi$x)),
              c(lo$y, rev(hi$y)),
              col = fill_cols[i], border = NA)

      est <- step_xy(t[valid], event[valid])
      lines(est$x, est$y, lwd = 2, lty = ltys[i], col = line_cols[i])
    }
  }
}

ltys <- c(1, 2)
line_cols <- c("#6cc0a8", "#5A4B62")

# ---- Build the 3-panel figure: P2 vs P3, P2 vs P4, P2 vs P5 ----
png("P2_vs_P3P4P5_Dist_3panel.png", width = 12, height = 8, units = "in", res = 300)
par(bg="white", mfrow = c(1,3), mar = c(4,4,3,1), mgp = c(2,0.7,0))

# P2 vs P3
dat23 <- subset(squiddist, Trial %in% c("P2","P3"))
plot_pair_km(dat23, trial_levels = c("P2","P3"), main_title = "P2 vs P3")
  legend("topleft",
         legend = c("P2: White Light Baseline","P3: Opposite Start Location"),
         lty = ltys, lwd = 2, col = line_cols,
         bty = "n", cex = 0.9)

# P2 vs P4
dat24 <- subset(squiddist, Trial %in% c("P2","P4"))
plot_pair_km(dat24, trial_levels = c("P2","P4"), main_title = "P2 vs P4")
  legend("topleft",
         legend = c("P2: White Light Baseline","P4: Rearrange Landmarks"),
         lty = ltys, lwd = 2, col = line_cols,
         bty = "n", cex = 0.9)

# P2 vs P5
dat25 <- subset(squiddist, Trial %in% c("P2","P5"))
plot_pair_km(dat25, trial_levels = c("P2","P5"), main_title = "P2 vs P5")
  legend("topleft",
         legend = c("P2: White Light Baseline","P5: Remove Beacon"),
         lty = ltys, lwd = 2, col = line_cols,
         bty = "n", cex = 0.9)
```

```r
dev.off()
```

```
## pdf
##   2
```

Testing proportion escape within trial — logistic regression

This final analysis takes a different approach: instead of modeling *time to escape* (survival), it models the **binary outcome** of whether the squid escaped at all within the 10-minute trial. For each procedure (P1–P5), a **mixed-effects logistic regression** (GLMM with binomial family) is fit: escape success ~ Run + random intercept for Squid.

This addresses a slightly different question: does the *probability* of escaping (rather than the speed of escape) increase with practice across runs? A positive coefficient for Run means the squid is increasingly likely to escape at all — not just faster.

Note: this uses `squiddist` (distance data) rather than `squidtime` (time data) because the escape column in `squiddist` uses a cleaner `Y/N` coding with fewer missing values.

```r
escape_models <- list()

for(tr in paste0("P", 1:5)) {

  dat <- subset(
    squiddist,
    Trial == tr &
    !is.na(Escape..Y.N.) &
    !is.na(Run) &
    !is.na(Squid)
  )

  dat$escaped <- ifelse(dat$Escape..Y.N. == "Y", 1, 0)
  dat$Run <- as.numeric(dat$Run)

  escape_models[[tr]] <- glmer(
    escaped ~ Run + (1 | Squid),
    family = binomial,
    data = dat
  )
}
```

Summary: Escape frequency across runs

This table reports the effect of Run on the probability of escaping (binary outcome) from the mixed-effects logistic regression. The slope is the logistic regression coefficient for Run on the log-odds scale; a positive slope indicates that escape probability increases with repeated runs. An asterisk after the p-value indicates significance at the 0.05 level.

```r
escape_pvals <- sapply(escape_models, function(m) summary(m)$coefficients["Run", "Pr(>|z|)"])
escape_sig <- !is.na(escape_pvals) & escape_pvals < 0.05
```

```r
table1 <- data.frame(
  Procedure = paste0("P", 1:5),
  Slope = sapply(escape_models, function(m) round(summary(m)$coefficients["Run", "Estimate"], 3)),
  P_value = ifelse(is.na(escape_pvals), "NA",
                   ifelse(escape_pvals < 0.001, "<0.001",
                          formatC(escape_pvals, format = "f", digits = 3))),
  row.names = NULL
)

table1$P_value <- cell_spec(table1$P_value, format = "latex", bold = escape_sig)
table1$P_value <- ifelse(escape_sig, paste0(table1$P_value, "*"), table1$P_value)

kbl(
  table1[, c("Procedure", "Slope", "P_value")],
  col.names = c("Procedure", "Slope", "P-value"),
  format = "latex",
  escape = FALSE,
  caption = "Effect of Run on escape frequency (binary logistic regression).",
  align = c("l", "c", "c")
) |>
  kable_styling(full_width = FALSE, latex_options = "hold_position") |>
  column_spec(1, bold = TRUE)
```

Table 4: Effect of Run on escape frequency (binary logistic regression).

Procedure	Slope	P-value
P1	0.132	0.718
P2	-0.101	0.654
P3	-0.840	**0.004***
P4	-0.568	**0.029***
P5	-0.324	0.218